Level 1: The Warm-Ups

1. The "Hype Man"

The Goal: Write a function that acts as your personal hype man.

- **Requirements:**
  - Create a function called `hype_man` that takes two parameters: `name` and `skill`.
  - The function should `print` a highly energetic sentence using both arguments.
  - **Example Call:** `hype_man("Alex", "coding")`
  - **Expected Output (Printed):** "Make some noise for Alex, the absolute master of coding!"

2. The Digital Bouncer

The Goal: Write a function that checks if someone is old enough to ride a roller coaster.

- **Requirements:**
  - Create a function called `check_age` that takes one parameter: `age`.
  - If the age is 12 or older, `print` the string "Enjoy the ride!"
  - If the age is under 12, `print` the string "Sorry, maybe next year."
  - **Example Call:** `check_age(15)`
  - **Expected Output (Printed):** "Enjoy the ride!"

Level 2: Data Structures & Loops

3. The Vowel Vacuum

The Goal: Write a function that sucks all the vowels out of a string and displays what's left.

- **Requirements:**
  - Create a function called `vowel_vacuum` that takes one parameter: `text`.
  - Inside the function, loop through the string. Keep only the consonants and spaces, and `print` the new, vowel-free string at the very end.
  - **Example Call:** `vowel_vacuum("Python is awesome")`
  - **Expected Output (Printed):** "Pythn s wsm"
  - *Hint: Create an empty string `result = ""` and add characters to it one by one if they aren't in "aeiouAEIOU", then print `result` outside the loop.*

4. The Splurge Calculator

The Goal: Find out how much money you just spent on a shopping spree and print the receipt.

- **Requirements:**
  - Create a function called `calculate_total` that takes one parameter: `cart_prices` (a list of numbers).
  - Use a loop to add up all the numbers in the list.
  - If the total is over $100, apply a $10 discount.
  - `print` a message with the final total.
  - **Example Call:** `calculate_total([25.50, 40.00, 50.00])`
  - **Expected Output (Printed):** "Your total today is $105.5"

Level 3: Dictionary & Set Integration

5. The Emoji Translator

The Goal: Replace specific words in a sentence with emojis using a dictionary and print the translation.

- **Requirements:**
  - Create a function called `emojify` that takes two parameters: `sentence` (a string) and `emoji_dict` (a dictionary).
  - Convert the sentence into a list of words.
  - Loop through the words. If a word is a key in the dictionary, replace it with the emoji value.
  - Join the words back together into a single string and `print` it.

Example Call:
my_dict = {"happy": "😊", "pizza": "🍕", "python": "🐍"}
emojify("I am so happy to eat pizza and code python", my_dict)

  -
  - **Expected Output (Printed):** "I am so 😊 to eat 🍕 and code 🐍"

6. The Unique Ingredient Finder

The Goal: You have two recipes, but you only want to print a single list of all the *unique* ingredients needed to make both.

- **Requirements:**
  - Create a function called `print_shopping_list` that takes two parameters: `recipe1` (a list of strings) and `recipe2` (a list of strings).
  - Convert the lists to sets to easily combine them and remove duplicates.
  - Convert the combined set back into a list and `print` it.
  - **Example Call:** `print_shopping_list(["flour", "sugar", "eggs"], ["eggs", "butter", "sugar", "vanilla"])`

- **Expected Output (Printed):** `['flour', 'sugar', 'eggs', 'butter', 'vanilla']` *(Note: order doesn't matter since sets are unordered)*.

Level 4: The Boss Fight

7. The Anagram Detective

The Goal: Write a function that determines if two words are anagrams of each other and announces the verdict.

- **Requirements:**
  - Create a function called `check_anagram` that takes two parameters: `word1` and `word2`.
  - The function must ignore capital letters and spaces.
  - `print` "Yes, those are anagrams!" if they match, and `print` "Nope, not an anagram." if they don't.
  - **Example Call:** `check_anagram("Clint Eastwood", "Old West Action")`
  - **Expected Output (Printed):** "Yes, those are anagrams!"